

PROJETO 4: MINIX

Gustavo Borges Peres da Silva¹, Michael P. C. Silva¹, Rafael S. Melo¹

¹Departamento de Ciência da Computação – Universidade Federal de Catalão (UFCAT)

{gustavo.silva, michael.silva}@discente.ufcat.edu.br,
rafinhalelograma@hotmail.com

Objetivos

- Familiarizar o aluno com o SO MINIX e testes em ambiente de máquina virtual.
- Processos órfãos e zumbis.

1. TAREFA 6 (gerenciamento de processos): Processos zumbis (zombie.c)

No Unix e SOs Unix-like, um processo zumbi ou defunto é um processo que completou sua execução mas ainda possui uma entrada na tabela de processos. Esta entrada ainda é necessária para permitir ao processo que iniciou o processo (agora zumbi) a leitura de seu status de saída. O termo processo zumbi deriva da definição comum de zumbi - uma pessoa não morta.

Nos termos desta metáfora, o processo filho já morreu, mas ainda não foi enterrado.

Quando um processo termina, toda a memória e recursos associados a ele são deslocados de modo que possam ser usados por outros processos. Entretanto, a entrada do processo na tabela de processo ainda perdura. O pai pode ler o status de saída do filho pela execução da chamada de sistema wait, momento no qual o zumbi é removido. A chamada wait pode ser executada em código sequencial, mas é comumente executada em um handler pelo sinal SIGCHLD, o qual o pai recebe sempre que um filho morre.

Após a remoção do zumbi, seu ID de processo e entrada na tabela de processo pode então ser reutilizado. Entretanto, se um pai falha na chamada wait, o zumbi será deixado na tabela de processo. Em algumas situações, isto pode ser desejado, por exemplo, se o pai cria outro processo filho, ele se assegura que não irá alocar o mesmo ID de processo.

Um processo zumbi não é o mesmo que um processo órfão. Um processo órfão é um processo que ainda está sendo executado, mas cujos pais já morreram. Eles não se tornam processos filhos; ao invés disso, eles são adotados pelo init (ID de processo 1), o qual espera por seus filhos.

Zumbis podem ser identificados na saída do comando Unix 'ps' pela presença de um "z" na coluna STAT. Zumbis que existem por mais do que um pequeno intervalo de tempo, tipicamente indicam um bug no programa do processo pai, ou apenas uma decisão incomum ao sepultar filhos (veja o exemplo Figura 1). A presença de uns poucos zumbis não é algo preocupante em si, mas pode indicar um problema que se tornaria sério sob cargas excessivas. Como não existe memória alocada a processos zumbis, exceto para a tabela de processo, a preocupação primária com muitos zumbis não é ficar sem memória, mas ao invés disso, ficar sem números para o ID do processo.

Para remover zumbis de um sistema, o sinal SIGCHLD pode ser enviado ao pai manualmente, usando o comando kill. Se o processo pai ainda se recusa a encerrar o zumbi, o próximo passo seria remover o processo pai. Quando um processo perde o seu pai, o init se torna o seu novo pai. O init periodicamente executa a chamada de sistema wait enterrando quaisquer zumbis que o possuem como pai.

```
$ ps -f
```

S	PID	PPID	CMD
S	311	129	zombie
Z	312	311	<defunct>
Z	313	311	<defunct>
Z	314	311	<defunct>
Z	315	311	<defunct>
Z	316	311	<defunct>

Figura 1 – exemplo processos zumbi.

```
#include <sys/wait.h>
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>

int main(void) {
    pid_t pids[5];
    int i;
    printf("PAI: Aqui esta meu PID: %d\n", getpid());
    for (i = 4; i >= 0; --i) {
        pids[i] = fork();
        if (pids[i] == 0) {
            printf("FILHO: O PID do meu PAI eh: %d\n", getppid());
            printf("FILHO: Meu PID eh: %d\n", getpid());
            _exit(0);
        }
    }
    for (i = 4; i >= 0; --i)
        waitpid(pids[i], NULL, 0);
    return 0;
}
```

Código 1 - Conteúdo do arquivo zombie.c

Sendo assim, nessa tarefa, foi solicitado pelo professor, de realizarem modificações no arquivo zombie.c (Código 1) de modo que este forneça uma saída similar a apresentada para o comando ps de acordo com a Figura 1 na qual são apresentadas apenas as entradas de interesse.

Além disso, foi solicitado também para explicarmos as relações de parentesco entre os processos 311,312,313,314,315 e 316, e quem é o pai de Zombie.

1.1. Solicitação 1: modificação no arquivo zombie.c

Para este experimento, substituímos o laço do final do código que antes servia para esperar que todos os processos filhos terminassem por uma linha que realizasse uma pausa de 10 segundos antes de terminar o processo pai. Isso significa que o processo pai permanecerá ativo por 10 segundos após a conclusão de todos os processos filhos antes de terminar, permitindo-nos usar o comando **ps -f** e ver os processos (Código 2).

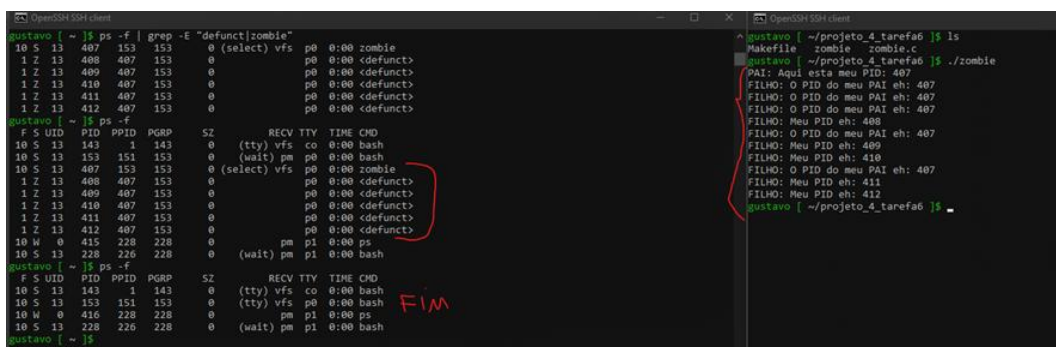
Além disso, para uma visualização mais específica ainda, usamos o seguinte comando:

ps -f | grep -E "defunct|zombie", isso serve para listar apenas o que o professor solicitou (Figura 2).

```
#include <sys/wait.h>
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>

int main(void) {
    pid_t pids[5];
    int i;
    printf("PAI: Aqui esta meu PID: %d\n", getpid());
    for (i = 4; i >= 0; --i) {
        pids[i] = fork();
        if (pids[i] == 0) {
            printf("FILHO: O PID do meu PAI eh: %d\n", getppid());
            printf("FILHO: Meu PID eh: %d\n", getpid());
            _exit(0);
        }
    }
    sleep(10); // Pausa de 10 segundos
    return 0;
}
```

Código 2 - Atualização do arquivo zombie.c



```
gustavo [ ~ ]$ ps -f | grep -E "defunct|zombie"
10 S 13 407 153 153 0 (select) vfs p0 0:00 zombie
1 Z 13 408 407 153 0 p0 0:00 <defunct>
1 Z 13 409 407 153 0 p0 0:00 <defunct>
1 Z 13 410 407 153 0 p0 0:00 <defunct>
1 Z 13 411 407 153 0 p0 0:00 <defunct>
1 Z 13 412 407 153 0 p0 0:00 <defunct>
gustavo [ ~ ]$ ps -f
F S UID PID PPID PGRP SZ RECV TTY TIME CMD
10 S 13 143 1 143 0 (tty) vfs co 0:00 bash
10 S 13 153 151 153 0 (wait) pm p0 0:00 bash
10 S 13 407 153 153 0 (select) vfs p0 0:00 zombie
1 Z 13 408 407 153 0 p0 0:00 <defunct>
1 Z 13 409 407 153 0 p0 0:00 <defunct>
1 Z 13 410 407 153 0 p0 0:00 <defunct>
1 Z 13 411 407 153 0 p0 0:00 <defunct>
1 Z 13 412 407 153 0 p0 0:00 <defunct>
10 W 0 415 228 228 0 pm p1 0:00 ps
10 S 13 228 226 228 0 (wait) pm p1 0:00 bash
gustavo [ ~ ]$ ps -f
F S UID PID PPID PGRP SZ RECV TTY TIME CMD
10 S 13 143 1 143 0 (tty) vfs co 0:00 bash
10 S 13 153 151 153 0 (tty) vfs p0 0:00 bash
10 W 0 415 228 228 0 pm p1 0:00 ps
10 S 13 228 226 228 0 (wait) pm p1 0:00 bash
gustavo [ ~ ]$

gustavo [ ~/projeto_4_tarefa6 ]$ ls
Makefile  zombie  zombie.c
gustavo [ ~/projeto_4_tarefa6 ]$ ./zombie
PAI: Aqui esta meu PID: 407
FILHO: O PID do meu PAI eh: 407
FILHO: O PID do meu PAI eh: 407
FILHO: O PID do meu PAI eh: 407
FILHO: O PID do meu PAI eh: 407
FILHO: Meu PID eh: 408
FILHO: O PID do meu PAI eh: 407
FILHO: Meu PID eh: 409
FILHO: Meu PID eh: 410
FILHO: O PID do meu PAI eh: 407
FILHO: Meu PID eh: 411
FILHO: Meu PID eh: 412
gustavo [ ~/projeto_4_tarefa6 ]$
```

Figura 2 – Resultado do experimento com processos zombies no MINIX.

O comando **ps -f** mostra que temos cinco processos zumbis (defuntos) com PIDs 408, 409, 410, 411, 412, todos com PPID 407 (o processo pai que terminou).

Os processos zumbis são processos que terminaram, mas ainda têm uma entrada na tabela de processos. Eles não estão mais em execução, mas ainda têm um status de "defunto" até que o processo pai leia seu status de saída. Como o processo pai terminou, esses processos zumbis foram adotados pelo processo init (com PID 1), que está esperando por eles.

Se a gente quiser se livrar desses processos zumbis, fizemos um arquivo sigchld.c (Código 3) no qual usamos o sinal SIGCHLD para lidar com o término dos processos filhos. O sinal SIGCHLD é enviado para o processo pai quando um dos seus processos filhos termina ou é interrompido. O processo pai usa a função `signal()` para configurar um manipulador de sinal para o sinal SIGCHLD. O manipulador de sinal é uma função chamada `handle_sigchld()` que é chamada quando o sinal SIGCHLD é recebido. A função `handle_sigchld()` chama a função `waitpid()` para esperar que um processo filho termine e imprime uma mensagem indicando que o processo do filho terminou.

O uso do sinal SIGCHLD é útil porque permite que o pai seja notificado quando um processo filho terminou, sem precisar esperar ativamente pelo término de cada processo filho. Isso é especialmente útil quando o processo pai precisa lidar com um grande número de processos filhos ou quando os processos filhos podem terminar em momentos diferentes.

```
#include <sys/wait.h>
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handle_sigchld(int sig) {
    pid_t pid;
    int status;
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        printf("PAI: Processo filho com PID %d terminou.\n", pid);
    }
}

int main(void) {
    pid_t pids[5];
    int i;
    printf("PAI: Aqui esta meu PID: %d\n", getpid());

    // Configura o sinal SIGCHLD para ser tratado pela função handle_sigchld
    signal(SIGCHLD, handle_sigchld);

    for (i = 4; i >= 0; --i) {
        pids[i] = fork();
        if (pids[i] == 0) {
            sleep(2); // Pausa de 2 segundos
            printf("FILHO: O PID do meu PAI eh: %d\n", getppid());
        }
    }
}
```

```

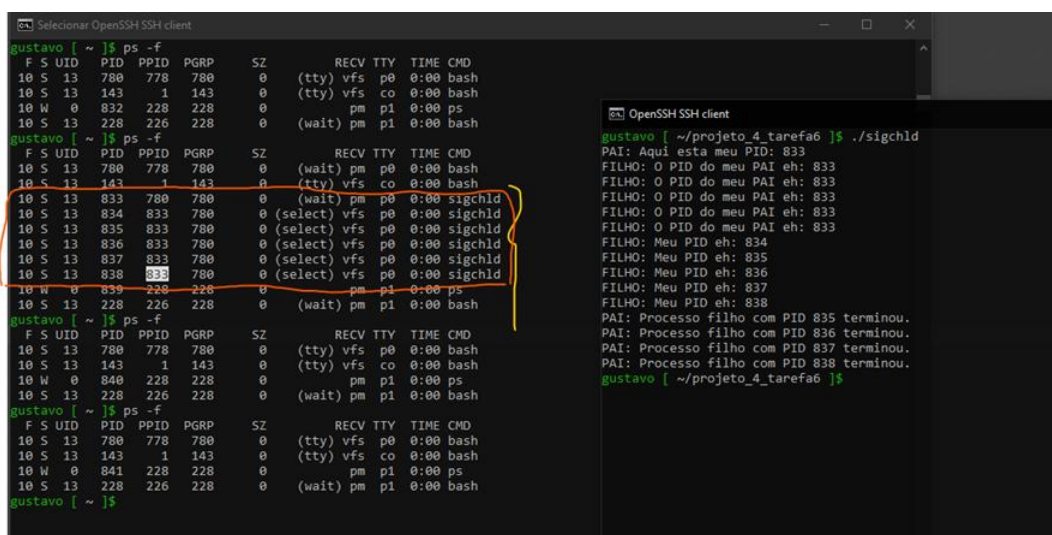
        printf("FILHO: Meu PID eh: %d\n", getpid());
        _exit(0);
    }
}

// Espera pelos processos filhos antes de terminar
for (i = 4; i >= 0; --i) {
    waitpid(pids[i], NULL, 0);
}

return 0;
}

```

Código 3 - Conteúdo do arquivo sigchld.c.



```

gustavo [ ~ ]$ ps -f
F S UID PID PPID PGRP SZ RECV TTY TIME CMD
10 S 13 780 778 780 0 (tty) vfs p0 0:00 bash
10 S 13 143 1 143 0 (tty) vfs co 0:00 bash
10 W 0 832 228 228 0 (wait) pm p1 0:00 ps
10 S 13 228 226 228 0 (wait) pm p1 0:00 bash
gustavo [ ~ ]$ ps -f
F S UID PID PPID PGRP SZ RECV TTY TIME CMD
10 S 13 780 778 780 0 (wait) pm p0 0:00 bash
10 S 13 143 1 143 0 (tty) vfs co 0:00 bash
10 S 13 833 780 780 0 (select) vfs p0 0:00 sigchld
10 S 13 834 833 780 0 (select) vfs p0 0:00 sigchld
10 S 13 835 833 780 0 (select) vfs p0 0:00 sigchld
10 S 13 836 833 780 0 (select) vfs p0 0:00 sigchld
10 S 13 837 833 780 0 (select) vfs p0 0:00 sigchld
10 S 13 838 833 780 0 (select) vfs p0 0:00 sigchld
10 W 0 839 228 228 0 (wait) pm p1 0:00 ps
10 S 13 228 226 228 0 (wait) pm p1 0:00 bash
gustavo [ ~ ]$ ps -f
F S UID PID PPID PGRP SZ RECV TTY TIME CMD
10 S 13 780 778 780 0 (tty) vfs p0 0:00 bash
10 S 13 143 1 143 0 (tty) vfs co 0:00 bash
10 W 0 840 228 228 0 (wait) pm p1 0:00 ps
10 S 13 228 226 228 0 (wait) pm p1 0:00 bash
gustavo [ ~ ]$ ps -f
F S UID PID PPID PGRP SZ RECV TTY TIME CMD
10 S 13 780 778 780 0 (tty) vfs p0 0:00 bash
10 S 13 143 1 143 0 (tty) vfs co 0:00 bash
10 W 0 841 228 228 0 (wait) pm p1 0:00 ps
10 S 13 228 226 228 0 (wait) pm p1 0:00 bash
gustavo [ ~ ]$

gustavo [ ~/projeto_4_tarefa6 ]$ ./sigchld
PAI: Aqui esta meu PID: 833
FILHO: O PID do meu PAI eh: 833
FILHO: O PID do meu PAI eh: 833
FILHO: O PID do meu PAI eh: 833
FILHO: O PID do meu PAI eh: 833
FILHO: O PID do meu PAI eh: 833
FILHO: Meu PID eh: 834
FILHO: Meu PID eh: 835
FILHO: Meu PID eh: 836
FILHO: Meu PID eh: 837
FILHO: Meu PID eh: 838
PAI: Processo filho com PID 835 terminou.
PAI: Processo filho com PID 836 terminou.
PAI: Processo filho com PID 837 terminou.
PAI: Processo filho com PID 838 terminou.
gustavo [ ~/projeto_4_tarefa6 ]$

```

Figura 3: resultado do experimento com uso de SIGCHLD.

Neste código 3, o processo pai cria cinco processos filhos e, em seguida, entra em um loop que chama a função `waitpid()` para esperar que cada processo filho termine. O processo pai continua a executar outras tarefas enquanto espera que os seus filhos terminem. Quando um processo filho termina, o processo pai é notificado pelo sinal `SIGCHLD` e o manipulador de sinal `handle_sigchld()` é chamado. O manipulador de sinal chama a função `waitpid()` para esperar que o processo filho termine e imprime uma mensagem indicando que o processo do filho terminou.

O uso do sinal `SIGCHLD` permite que o pai seja notificado quando um processo filho termina, sem precisar esperar ativamente pelo término de cada processo filho. Isso permite que o processo pai continue a executar outras tarefas enquanto espera que os processos filhos terminem.

1.2. Solicitação 2: explicação de relação parentesco

Em relação a Figura 1, o comando `ps -f` fornecido mostra que os processos que vão de 312 a 316 são filhos do processo zombie (defunct), como indicado pelo estado Z na coluna S. Por outro lado, o processo 311 é o processo zombie (pai), como indicado pelo número 311 na coluna PPID (Parent Process ID).

Agora, em nossos experimentos, em relação a Figura 2, a saída do comando `ps -f` mostra que o processo 407 é o processo zombie (pai) e que os processos 408, 409, 410, 411 e 412 são filhos do processo zombie. Isso significa que o processo zombie é o pai desses processos defunct.

Por fim, em relação a Figura 3, no experimento com SIGCHLD, cinco processos filhos são criados e o processo pai aguarda cada um deles antes de encerrar, evitando que se tornem zumbis. A saída do comando `ps -f` revela que o processo pai (PID 833) possui cinco processos filhos (PIDs 834, 835, 836, 837 e 838), todos com o nome `sigchld`. Isso ocorre porque o sinal SIGCHLD é tratado pela função `handle_sigchld`, que imprime uma mensagem quando um processo filho termina. A saída do programa também mostra que os processos filhos imprimem corretamente o PID do pai e seu próprio PID. Além disso, a mensagem "PAI: Processo filho com PID XXX terminou." é exibida quando cada processo filho termina, indicando que o processo pai está aguardando corretamente pelos processos filhos.

2. Referências

[1] Dalton Matsuo Tavares, (2025) "PROJETO 4 – MINIX", material disponibilizado no google classroom e SIGAA, Janeiro.